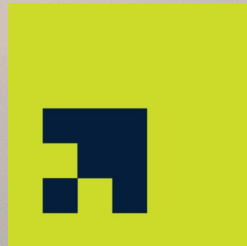


File Handling

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences



Data for your program

- Data source:

- Input device, e.g., Keyboard or camera
- Web scraping
- Data files
 - Raw Data: Binary or Text files
 - Processed/organized Data: Database or JSON

- Typically, dealing with files is essential for all programmers



- The operating system is important

- Set of elements that manage
 - The processes that run on the CPU
 - How memory is utilized and managed
 - How peripheral devices are used (such as printers and scanners)
- It allows the computer system to communicate with other systems
- It also provides support for the file system used

- File System allows programs to store data

- The File Management System is responsible for managing the [creation, access, and modification](#) of the storage of data in files
 - This data may be stored locally or remotely on disks, tapes, DVD drives, USB drives etc.



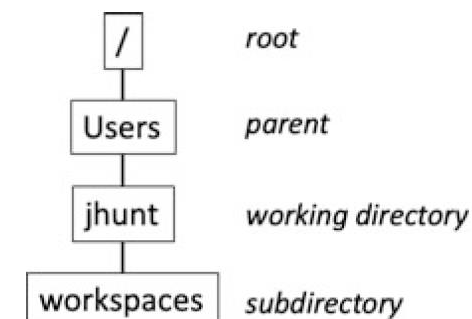
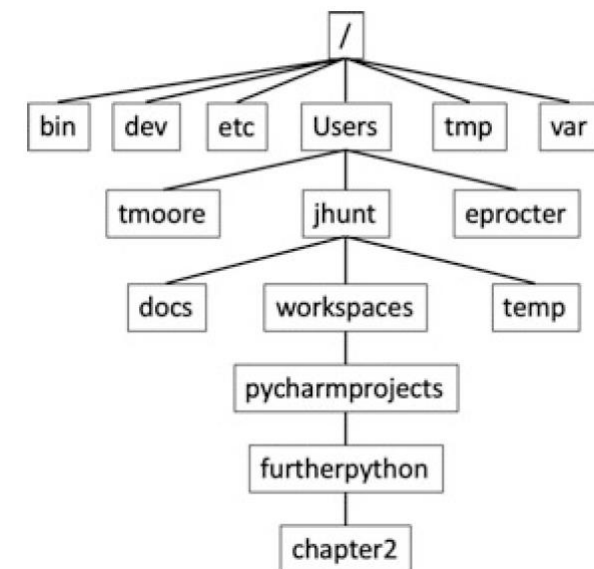
File Handling

[Hunt 2019]

4

- Modern operating systems organize files into a hierarchical structure

- Usually in the form of an inverted tree
- For example, in Linux, the root of the directory structure is shown as '/'
 - This root directory holds six subdirectories
 - In turn the Users subdirectory holds 3 further directories
- Each file is contained within a directory
- For any given directory, there are relationships with other directories
 - The root directory is the starting point for the hierarchical directory tree structure
 - A child directory of a given directory is known as a subdirectory
 - The directory that holds the given directory is known as the parent directory
 - The current working directory is the directory within which the program or user is currently working

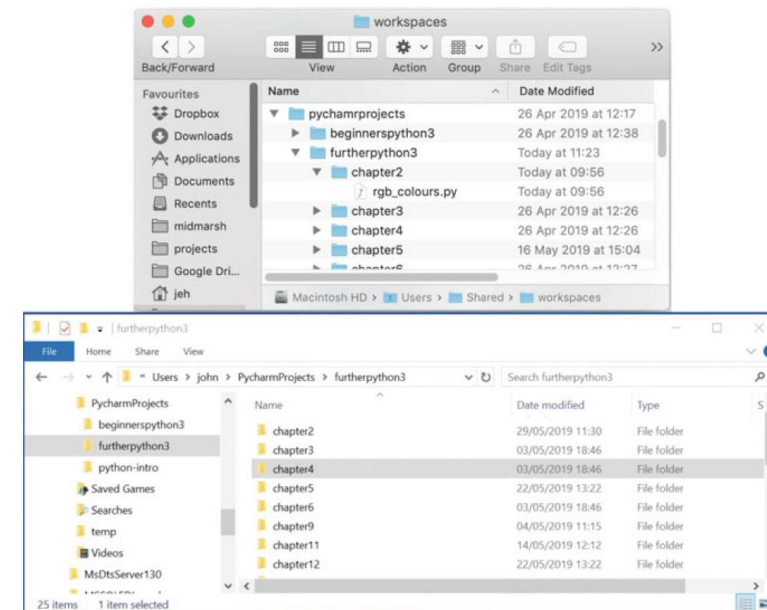




File Handling

- A user or a program can check the directory structure using either:
 - Command line (CL) using a set of commands at the terminal or command window
 - E.g., list files (`ls`), change directory (`cd`) or print the working directory (`pwd`) command
 - Graphical User Interfaces (GUIs) using the file manager/explorer application that allows a user to view the files

```
mohammad@GEDS35W10L10:/home$ pwd
/home
mohammad@GEDS35W10L10:/home$ cd mohammad
mohammad@GEDS35W10L10:~$ pwd
/home/mohammad
mohammad@GEDS35W10L10:~$ |
```





File Attributes

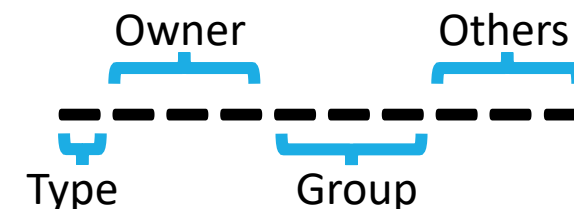
- A file has a set of attributes associated with it
 - Such as the date that it was created, the date it was last updated/modified, and the file size
 - Also an attribute indicating the owner of the file and permissions
 - Permissions can control who can read, write, or execute the file

```
mohammad@GEDS35W10L10:/mnt/c/Users/mmahdavi/Desktop/temp$ ls -l
total 496
-rwxrwxrwx 1 mohammad mohammad 40726 Sep 21 15:20 photo.png
-rwxrwxrwx 1 mohammad mohammad 464096 Sep 21 15:21 report.pdf
-rwxrwxrwx 1 mohammad mohammad 11 Sep 21 16:07 text.txt
mohammad@GEDS35W10L10:/mnt/c/Users/mmahdavi/Desktop/temp$ |
```



File Attributes

- File systems use permissions and attributes
 - To regulate the level of interaction that system processes can have with files and directories
- Type
 - The file type
 - Not part of its permissions
- Owner
 - The permissions that the owner has over the file
- Group
 - The permissions that the group has over the file
- Others
 - The permissions that all the other users have over the file





File Attributes

[Hunt 2019]

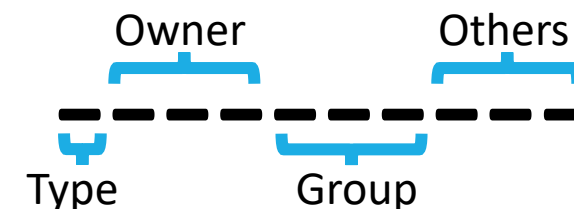
8

- The file owner can have rights specified for reading, writing and executing a file

- These are usually represented by the symbols 'r', 'w', and 'x' respectively

- **-rwx-----**

- Here the first dash is left blank as it is to do with special files (or directories)
 - The next set of three characters represents the permissions for the owner
 - The file owner is allowed to read, write, and execute a file
 - However, the next six characters are all dashes indicating that the group and all other users cannot access the file at all





File Attributes

[Hunt 2019]

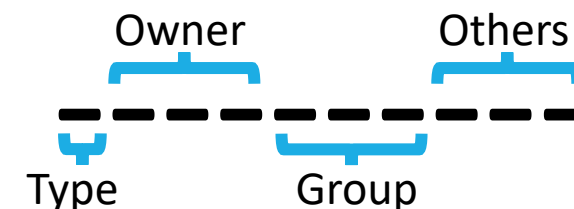
9

- The group that a file belongs to is a group that can have any number of users as members

- A member of the group will have the access rights as indicated by the group settings on the file
- As for the owner of a file, these can be to read, write, or execute the file

- `----r-x---`

- Here, this example indicates that only members of the group can read and execute the file
- Note that group members cannot write the file
 - They therefore cannot modify the file



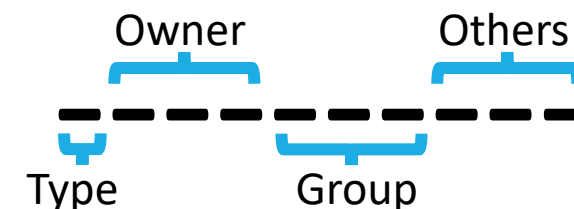


File Attributes

[Hunt 2019]

11

- Of course, a file can mix the above permissions together
- `-rwxr-xr--`
 - An owner may be allowed to read, write, and execute a file
 - The group may be able to read and execute the file
 - All other users can only read the file

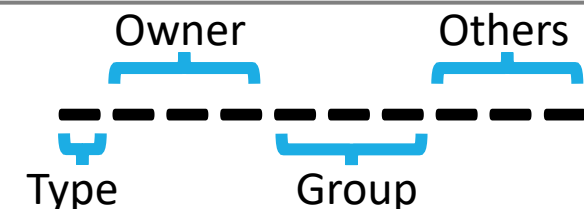




• There is also a numeric notation that is used with Unix style systems

- The numeric notation uses three digits to represent the permissions
- Each of the three rightmost digits represents a different component of the permissions
 - I.e., owner, group, and others
- Each of these digits is the sum of its component bits in the binary numeral system
- As a result, specific bits add to the sum as it is represented by a numeral
 - The read bit adds 4 to its total
 - In binary 100
 - The write bit adds 2 to its total
 - In binary 010
 - The execute bit adds 1 to its total
 - In binary 001

Symbolic notation	Numeric notation	Meaning
<code>rwX---</code>	<code>0700</code>	Read, write, and execute only for owner
<code>-rwxrwx---</code>	<code>0770</code>	Read, write, and execute for owner and group
<code>-rwxrwxrwx</code>	<code>0777</code>	Read, write, and execute for owner, group and others



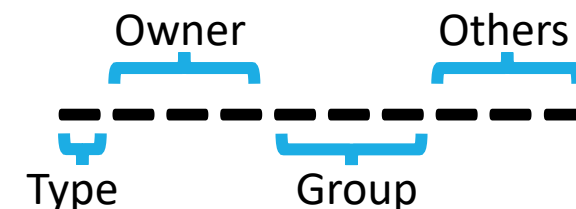


File Attributes

[Hunt 2019]

13

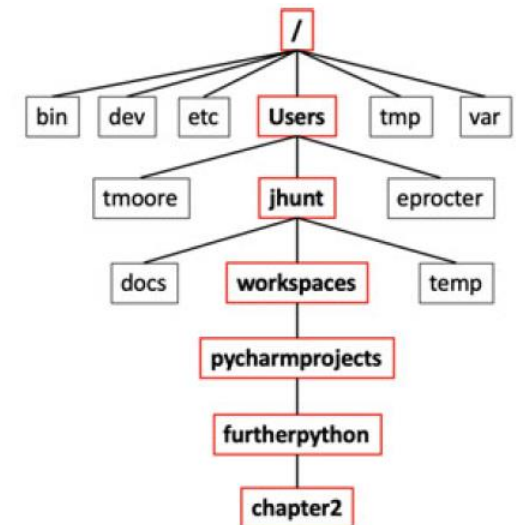
- Directories have similar attributes and access rights to files
- `dr-xr-x---`
 - The above symbolic notation indicates that a directory (indicated by the 'd') has read and execute permissions for the directory owner and for the group
 - Other users cannot access this directory





Paths

- A file path is a particular combination of directories that leads to a specific sub-directory or file
 - This concept is important as all file systems represent an inverted tree of directories and files
 - It is thus important to be able to uniquely reference locations with the tree
 - For example, in the following diagram, the path “/Users/jhunt/workspaces/pycharmprojects/furtherpython/chapter2” is highlighted



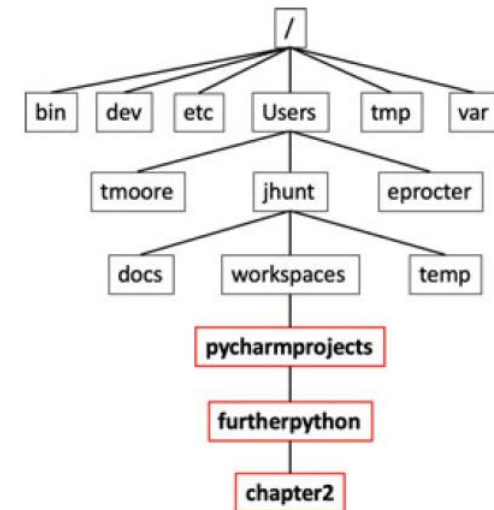


- A path may be absolute or relative

- An absolute path provides a complete sequence of directories from [the root to the specific subdirectory or file](#)
 - The absolute path will work wherever a program or user is currently located within the directory tree
- A relative path provides a sequence from the [current working directory to a particular subdirectory or file](#)
 - However, a relative path may only be relevant in a specific location
 - For example, starting from “pycharmprojects” folder, the relative path “pycharmprojects/furtherpython/chapter2”
 - The relative path will change with the starting folder (location)

- Note that

- An absolute path starts from the root directory
 - Represented by ‘/’
- A relative path starts from a particular subdirectory
 - Such as pycharmprojects





File Input/Output

[Hunt 2019]

17

- File Input/Output involves reading and writing data to and from files
 - It is often just referred to as File I/O
 - The data can be in different formats



File Input/Output

- A common format is the ASCII text format
 - ASCII stands for American Standard Code for Information Interchange
 - The ASCII format is a set of codes that represent various characters
 - That is widely used by operating systems
 - The following table illustrates some of the ASCII character codes and what they represent
- ASCII is a very useful format to use for text files
 - As they can be read by a wide range of editors and browsers



File Input/Output

- Programming languages such as Python use a different set of character encodings
 - Such as a Unicode character encoding (such as UTF-8)
 - It may be necessary to convert from ASCII into Unicode (e.g., UTF-8) and vice versa
 - When reading and writing ASCII files in Python
- Unicode is another standard for representing characters using various codes
 - Unicode encoding systems offer a wider range of possible character encodings than ASCII
 - For example, the latest version of Unicode in May 2019, Unicode 12.1, contains a repertoire of 137,994 characters covering 150 modern and historic scripts, as well as multiple symbol sets and emojis



File Input/Output

- Another option is a binary format for data
 - The advantage of using binary data is
 - Almost no conversion is required from the internal representation of the data used in the Python program into the format stored in the file
 - It is quicker for a program to read and write and uses less disk space
 - The downside of a binary format is that
 - It is not in a human-readable format
 - It may need special conversion for some other programming languages, such as Java or C#, to read the data in the files



• Reading from, and writing text files in Python is relatively straightforward

- The built-in `open()` function creates a [file object](#) to read and write data from a file
- All that you need:
 - File name
 - Indicates the file to be accessed
 - Relative or an absolute path is required
 - Access mode
 - E.g., read, write, or append
 - A complete list of possible values is given below in the table
 - This is an optional parameter, and the default file access mode is read (r)
 - buffering
 - If the buffering value is set to 0, no buffering takes place
 - If the buffering value is 1, line buffering is performed while accessing a file
 - It improves performance by grouping data reads together

Mode	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing

```
file_object = open(file_name, access_mode, buffering)
```



- The file object itself has several useful attributes

- “file.closed”
 - Returns True if the file has been closed
- “file.mode”
 - Returns the access mode with which the file was opened
- “file.name”
 - Returns the name of the file

```
file = open("myfile.txt", "w")  
print("file.name:", file.name)  
print("file.closed:", file.closed)  
print("file.mode:", file.mode)
```




- The “file.close()” method is used to close the file once you finished with it

- This will
 - Flush any unwritten information to the file
 - Remove the reference of the file object
 - This is important to do as leaving a reference to a file open can cause problems in larger applications
 - Resulting in future errors being thrown as files can no longer be opened
 - Consuming a lot of memory
 - Force operating system to write and save the file

```
file = open("myfile.txt", "w")

print("file.name:", file.name)
print("file.closed:", file.closed)
print("file.mode:", file.mode)

file.close()

print("file.closed now:", file.closed)

file.name: myfile.txt
file.closed: False
file.mode: w
file.closed now: True
```



- Reading data from a text file by using one of the following methods:
 - “read()”
 - Returns the entire contents of the file as a single string
 - “readline()”
 - Reads the next line of text from a file
 - Used to read a line at a time
 - It returns all the text from one line, including the newline character
 - “readlines()”
 - Returns a [list](#) of all the lines in a file
 - Each item in the list is a single line
- Reading line(s) from a file uses a pointer or counter
- Once you have read line(s) from a file, that line is not read again
 - Thus, using “readlines()” twice returns an empty list, whatever the contents of the file

```
f = open("myfile.txt", "r")  
content = f.read()  
print(content)
```



- Typically, the file's contents are processed line-by-line

- File iteration accesses each line in the file and makes that line available in the for-loop

- Example, print file contents

- The print function uses the end character as white space " " instead of a newline as the line string already possesses the newline character read from the file

```
file = open("myfile.txt", "r")
lines = file.readlines()
for line in lines:
    print(line, end=" ")
file.close()
```

```
file = open("myfile.txt", "r")
for line in file:
    print(line, end=" ")
file.close()
```



File Contents Iteration

- It is also possible to use the list comprehension

- To provide a very concise way to load and process lines in a file into a list
- It is similar to the effect of `readlines()`
 - But we are now able to preprocess the data before creating the list

```
file = open("myfile.txt", "r")
lines = [line.upper() for line in file]
file.close()
print(lines)
```



Writing Data to Files

- Writing a string to a file is supported by the “write()” method

- The file object must have an access mode to allow writing
 - Such as ‘w’
- Note, the write method does not add a newline character (represented as '\n') to the end of the string
 - You must do this manually
- The following code creates a new file called my-new-file.txt and write some text in it

```
f = open("my-new-file.txt", "w")  
f.write("Hello from Python!\n")  
f.write("Working with files is easy...\n")  
f.write("It is cool...\n")  
f.close()
```



Using Files and with Statements

- It is important to free resources after usage
- The file object can be used with the “with” statement
 - It will create the file object with the name used after the command “as”
 - It will close the file and remove the reference when the block of code is finished with

```
with open("my-new-file.txt", "r") as f:  
    lines = f.readlines()  
    for line in lines:  
        print(line, end="")
```



Sequential Access vs Random Access

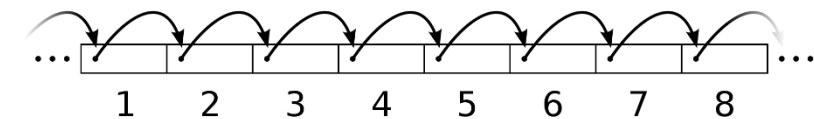
[Hunt 2019]

https://en.wikipedia.org/wiki/Random_access

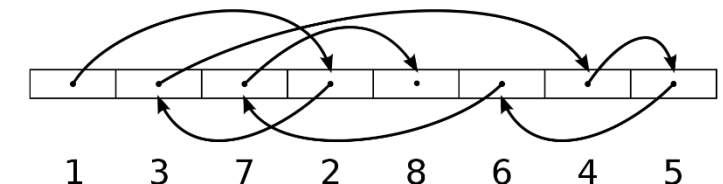
29

- Data can be read or written in two ways
 - Sequential access
 - Random access
- Previously, the access proposed was sequentially
 - The first line is read before the second, and so on
 - This is (probably) the most common approach
 - Also, it is possible to use a random-access approach to access a file

Sequential access



Random access





Sequential-Access Files

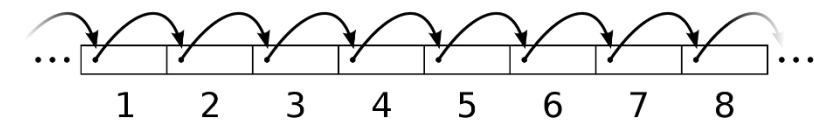
- Sequential access to data in a file means that the program accesses the file's data sequentially

- Starting at the beginning of a file and processing the data an item at a time until the end of the file is reached
- The read process only moves forward to the next item of data
- Write after the last written line

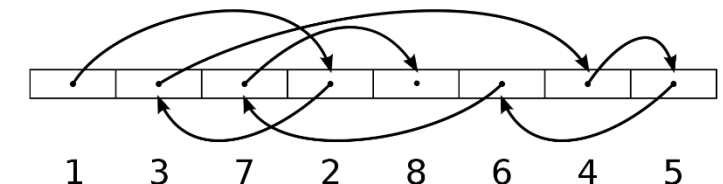
- Sequential file access has advantages

- When a program needs to access information in the same order
 - Each time the data is read
- It is also faster to read or write all the data sequentially than direct access
 - As there is no need to move the file pointer around

Sequential access



Random access





Random-Access Files

[Hunt 2019]

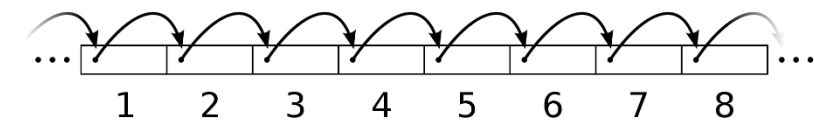
https://en.wikipedia.org/wiki/Random_access

31

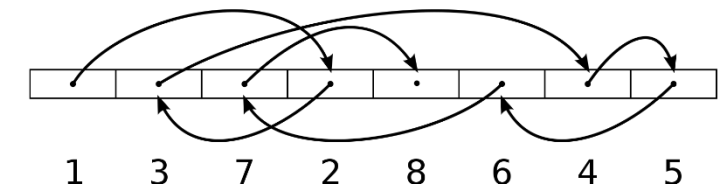
- Random Access means that the program can access file data anywhere, anytime

- The program can position the access pointer at any particular point in the file
- Then It can start read/ write at that point
 - If it is reading, then it will read the next data item relative to the pointer
 - If it is writing data, then it will write data from that point
 - Rather than at the end of the file
 - If there is already data at that point in the file, then it will be overwritten
- It is also known as Direct Access, as it goes directly to that location for the data
 - The computer program needs to know where the data is stored within the file

Sequential access



Random access





Random-Access Files

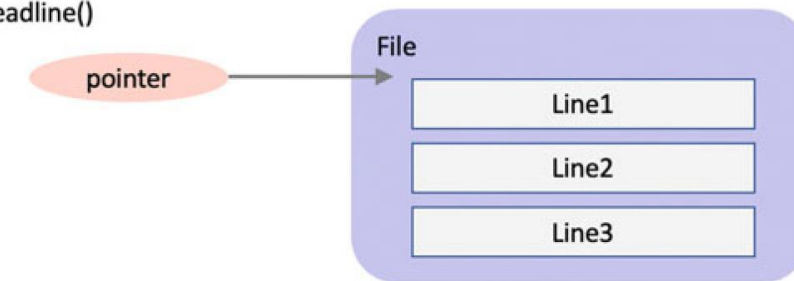
[Hunt 2019]

32

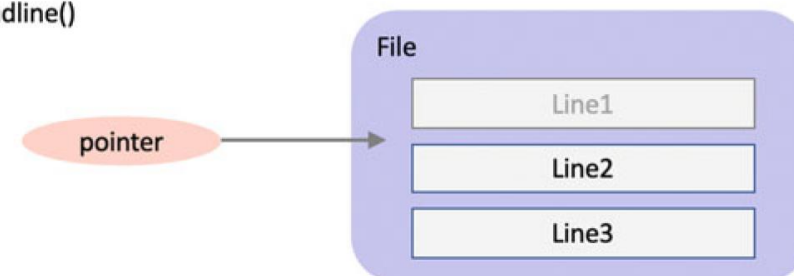
- We can maintain a pointer into a file

- To indicate where we are in that file in terms of reading or writing data
 - Before anything is read from a file the pointer is before the beginning of the file
 - Reading the first line of text would for example, advance the point to the start of the second line in the file
- The programmer manually moves the pointer to the location required and reads or writes text relative to that pointer
 - When randomly accessing the contents of a file
 - This means that they can move around in the file reading and writing data

Before readline()



After readline()





- The random-access aspect of a file is provided by the seek method of the file object

- `file.seek(offset, whence)`
 - This method determines where the next read or write operation (depending on the mode used in the `open()` call) takes place
 - The offset parameter indicates the position of the read/write pointer within the file
 - The optional whence parameter indicates where the offset is relative to
 - 0 indicates that the offset is relative to start of file (the default)
 - 1 means that the offset is relative to the current pointer position
 - 2 indicates the offset is relative to end of file

```
f = open("text.txt", "w")
f.write("abcdefghijklmnopqrstuvwxyz\n")
f.seek(10, 0)
f.write("HELLO")
f.seek(6, 0)
f.write("BOO")
f.close()
with open("text.txt", "r") as f:
    for line in f:
        print(line, end="")
```

abcdef**BOO**j**HELLO**pqrstuvwxyz



Sequential Access vs Random Access

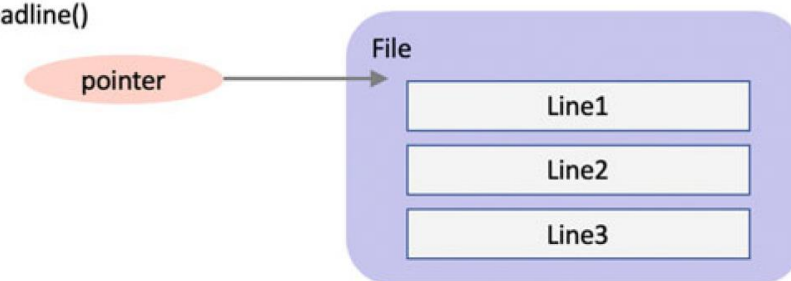
[Hunt 2019]

34

- Random access files have advantages

- They are more flexible as data does not need to be written or read in the order in which it is obtained
- It is also possible to jump to just the location of the data required and read that data
 - Rather than needing to sequentially read through all the data to find the data items of interest

Before readline()



After readline()

